Table des matières

Exercices corrigés 06 — Orchestrer et rattraper 1
Questions courtes 1
Questions pratiques 2
Diagnostics 3
Questions à développer 3
Mises en situation 3
Corrigé 5

Exercices corrigés 06 — Orchestrer et rattraper

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les exercices marqués [labo] portent sur les deux DAG et sur la fenêtre que vous venez d'écrire ; les autres vérifient que le cours est clair dans votre tête. Tout vient de la Théorie 06 et du Labo 06.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Le partage du travail. Nommez trois choses qu'un orchestrateur fait, et deux qu'il ne fait pas.
2. Sans cycle. Pourquoi un graphe de tâches doit-il être acyclique ? Que ne pourrait-on plus faire si une flèche revenait en arrière ?
3. Ce qu'une flèche dit. Une flèche va de `bronze` vers `raw`. Donnez deux affirmations qu'elle garantit, et deux qu'on lui prête à tort.
4. Les deux temps. Une exécution du DAG mensuel porte la date du premier février 2026 et tourne le 16 septembre 2026. Lequel des deux est l'intervalle de données, et lequel n'a aucune importance pour le contenu chargé ?
5. `{{ ds }}`. Que vaut cette valeur pour l'exécution du 15 mars du DAG quotidien, rejouée aujourd'hui ?
6. Trois mois de retard. Un DAG mensuel démarre avec une date de début remontant à trois mois. Combien d'exécutions le scheduler crée-t-il, et dans quel ordre les lance-t-il ?

© 2026 iAlexMG. Tous droits réservés.

7. La borne de fin. Un DAG mensuel doit couvrir janvier, février et mars 2026. À quelle date faut-il poser sa borne de fin, et qu'arrive-t-il si on la pose au premier avril ?
8. Deux filtres. La fenêtre et le journal des chargements écartent tous deux des fichiers. Qu'est-ce que chacun regarde, et pourquoi la fenêtre passe-t-elle en premier ?
9. Ce qui n'a pas de date. Le lot d'un export livré en entier vaut « complet ». Pourquoi la fenêtre le laisse-t-elle passer dans toutes les tranches, plutôt que de l'écarter ?
10. Une relivraison. Le lot `2023-03_bis` tombe-t-il dans une fenêtre du mois de mars 2023 ? Pourquoi ce choix, plutôt que l'inverse ?
11. Trois idempotences. Nommez les trois mécaniques qui rendent le pipeline de Boréal rejouable, et dites ce que chacune garantit.
12. Le bronze non fenêtré. Pourquoi l'étape `bronze` reconstruit-elle tout à chaque passage au lieu de se limiter à la journée de l'exécution ?
13. Rater, tarder, attendre. Associez chacune de ces trois situations à sa réponse : la nouvelle tentative, l'alerte de retard, le capteur.
14. Les SLA. Un tutoriel trouvé en ligne montre `sla=timedelta(minutes=30)` sur une tâche. Que faut-il en faire avec Airflow 3, et par quoi se remplace-t-il ?
15. Le pool. Qu'est-ce qu'un pool protège, que le nombre d'exécutions simultanées d'un DAG ne protège pas ?
16. La tâche vide. `dbt_build` ne fait rien. Donnez deux raisons de la poser quand même.

Questions pratiques

1. [labo] Une seule journée. Écrivez la commande qui charge la télémétrie du 2 février 2026, et rien d'autre. Dites combien de fichiers vous attendez.
2. [labo] Un trimestre d'un coup. Écrivez la commande qui charge l'ERP et les garanties du premier trimestre 2026 en une seule exécution, sans passer par Airflow.
3. [labo] Le profil des exécutions. Écrivez la requête qui donne, pour chaque lot de 2026, le nombre de fichiers chargés et le total des lignes, du plus ancien au plus récent.
4. [labo] La preuve, retournée. Sans relancer le pipeline, écrivez la requête qui vérifie qu'aucun fichier n'a été chargé deux fois sous la même empreinte dans la table des chargements.
5. [labo] Un DAG hebdomadaire. Écrivez la déclaration d'un DAG qui tournerait chaque lundi sur la semaine écoulée, avec la fenêtre correspondante. Les deux bornes doivent venir de la date logique de l'exécution.
6. [labo] La météo fenêtrée. L'étape `meteo` ignore la fenêtre et lit ses dates dans `config.toml`. Décrivez le changement qui la brancherait sur la fenêtre, et ce qu'il faudrait vérifier avant.

© 2026 iAlexMG. Tous droits réservés.

7. [labo] Un capteur. Le DAG mensuel ne devrait démarrer que si l'extrait `VBAK.csv` du mois est déposé. Nommez la tâche qui ferait cette attente, et dites où elle se placerait dans le graphe.

8. [labo] Le pool en action. Sans toucher aux DAG, écrivez la commande qui montre que le pool `entrepot` n'a qu'une place, puis celle qui lui en donnerait deux. Dites ce qui se passerait alors.

Diagnostics

1. Une collègue écrit `bash_command="python -m pipeline raw --depuis $(date +%F) --jusqua $(date +%F)"`. Le DAG tourne, ne rate jamais, et pourtant le rattrapage des trois mois a tout mis dans le même lot. Que s'est-il passé ?

2. Un DAG mensuel a `catchup=True` et une date de début au premier janvier 2022. À la première mise en route, le scheduler crée cinquante et une exécutions qui se bousculent. Deux choses sont à corriger, lesquelles ?

3. La tâche `raw` du DAG quotidien échoue une fois sur trois avec `Could not set lock on file ... Conflicting lock is held`. Elle passe toujours au second essai. Pourquoi la nouvelle tentative ne suffit-elle pas comme réponse ?

4. Un DAG n'apparaît pas dans l'interface, et le dossier `dags/` contient bien son fichier. L'auteur jure que le fichier est correct. Où regardez-vous, et dans quel ordre ?

5. Après avoir ajouté `--livraisons erp,garanties` au DAG mensuel, un ingénieur constate que la télémétrie de 2026 n'est plus chargée du tout. Il en conclut que la fenêtre est cassée. Où est l'erreur de raisonnement ?

6. Le DAG quotidien met cinq minutes de plus que d'habitude, et se termine vert. Les journaux de la tâche `corpus` ne montrent rien d'anormal. Où est passée la différence ?

Questions à développer

A. L'intervalle de données est présenté comme la notion la plus mal comprise d'Airflow. Expliquez pourquoi l'erreur qu'il cause est plus dangereuse qu'une panne, et proposez deux habitudes de travail qui la rendent improbable.

B. Le rattrapage est vérifié dans ce cours par une comparaison ligne par ligne avec un chargement d'un bloc. Un collègue trouve cela excessif, et propose de comparer seulement les comptes de lignes. Qu'est-ce que sa méthode laisserait passer ? Construisez un cas concret.

C. Le pipeline de Boréal porte trois mécaniques d'idempotence. Un architecte propose de n'en garder qu'une, la fenêtre, « pour simplifier ». Défendez ou attaquez cette proposition, en disant ce qu'on perdrait précisément dans chaque cas.

Mises en situation

© 2026 iAlexMG. Tous droits réservés.

1. Chez Boréal, l’équipe de l’usine veut son propre DAG, qui écrira dans l’entrepôt toutes les quinze minutes. On vous demande votre avis avant qu’ils l’écrivent. Que soulevez-vous, et que proposez-vous ?
2. Une panne de trois jours a arrêté les conteneurs. À la reprise, on vous demande s’il faut recharger à la main ou laisser l’orchestrateur faire. Que répondez-vous, et sur quoi vous appuyez-vous pour en être sûr ?
3. Un ingénieur d’une équipe voisine vous montre son DAG : quinze tâches, toutes en ligne, une flèche entre chacune. Il trouve que « ça se lit bien ». Que regardez-vous avant de lui répondre, et que lui dites-vous ?
4. La direction demande une alerte « dès que le chargement est en retard ». Le chargement quotidien prend d’ordinaire huit minutes, mais il a déjà pris quarante minutes un lundi de mars sans conséquence. Comment choisissez-vous le délai, et que répondez-vous si on vous demande de le mettre à dix minutes ?
5. Six mois après ce cours, la formation Analytics Engineering est terminée et la tâche `dbt_build` doit être remplie. On vous demande si elle doit rester dans le DAG quotidien ou vivre dans un DAG à elle. Donnez les deux options, leurs conséquences, et votre recommandation.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

Chaque réponse cite la section de `Theorie_06_DE.md` qui la fonde, ou l'étape du `Labo_06_DE.md` qui la produit. Les chiffres sont ceux d'un projet chargé jusqu'à mars 2026, après les deux DAG du labo. Les commandes du pipeline se tapent dans la fenêtre 1 du labo, depuis `Projet_Boreal`, après `set PYTHONPATH=01_de` ; celles d'Airflow dans la fenêtre 2, l'Ubuntu ouvert dans `plateforme`. Les requêtes se tapent dans l'interface web, une par cellule, après `ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);`, les tables nommées en entier ; l'entrepôt se détache (`DETACH entrepot;`) avant toute commande du pipeline.

Questions courtes

1. Il fait trois choses : décider quoi lancer et quand, faire respecter l'ordre des dépendances, et garder la trace de ce qui s'est passé, avec les états, les durées et les journaux. Il ne transforme aucune donnée, et il ne connaît rien de votre entrepôt : il lance une commande et regarde son code de sortie.

Théorie : `Theorie_06_DE.md`, section 1.

2. Sans la contrainte d'acyclicité, il n'existerait pas d'ordre de passage : à chaque tâche du cycle, le scheduler devrait attendre une tâche qui l'attend. On ne pourrait plus décider par où commencer, ni dire quand une exécution est finie.

Théorie : `Theorie_06_DE.md`, section 2.

3. Elle garantit que `raw` ne démarrera pas avant que `bronze` ait fini, et que `raw` ne démarrera pas du tout si `bronze` échoue. Elle ne dit pas que `raw` démarrera juste après, ni qu'une donnée passe de l'une à l'autre : si `raw` lit ce que `bronze` a écrit, c'est parce que les deux touchent le même entrepôt, pas à cause de la flèche.

Théorie : `Theorie_06_DE.md`, section 2.

4. Le premier février 2026 est l'intervalle de données : c'est la tranche que l'exécution doit traiter. Le 16 septembre est l'heure d'exécution, un accident de calendrier, qui ne doit apparaître nulle part dans ce qui est chargé.

Théorie : `Theorie_06_DE.md`, section 3.

5. `2026-03-15`, quel que soit le jour où on la rejoue. C'est exactement ce qui rend l'exécution rejouable : elle redemande sa propre journée.

Théorie : `Theorie_06_DE.md`, section 3.

6. Trois, une par intervalle manqué, lancées dans l'ordre chronologique. Au labo, la première porte le mois de janvier, la dernière celui de mars.

Théorie : `Theorie_06_DE.md`, section 4.

© 2026 iAlexMG. Tous droits réservés.

7. Au 31 mars 2026. La borne de fin est inclusive : posée au premier avril, elle fait naître une quatrième exécution, celle du mois d'avril. On la croirait rouge, puisque les sources n'ont pas ce mois : elle est verte. Le chargeur ne lève rien quand sa fenêtre ne retient aucun fichier, il rend zéro ligne, l'étape passe à « réussi » et le code de sortie vaut 0. Le DAG montre donc quatre exécutions vertes, dont une vide.

   C'est le cas que le Diagnostic 2 du Cours 05 apprend à redouter : une exécution qui réussit tous les jours sans rien charger. On ne s'en aperçoit que de deux façons. En lisant les lignes écrites, dans `pipeline.executions`, plutôt que le seul état. Ou en déclarant ce qu'on attend d'une exécution, ce que le Cours 11 fera avec les contrats. L'orchestrateur, lui, ne le dira jamais : il sait si la commande a rendu 0, pas si elle a fait son travail.

Théorie : `Theorie_06_DE.md`, section 3.

8. La fenêtre regarde des noms de fichiers : elle décide si le lot d'un fichier tombe dans la tranche, sans rien lire. Le journal regarde des empreintes : il doit lire chaque fichier restant pour calculer son empreinte SHA-256. Filtrer par la fenêtre d'abord évite de relire 1 572 fichiers pour en charger 14.

Théorie : `Theorie_06_DE.md`, section 5.

9. Parce qu'un export livré en entier n'a pas de date : l'écarter reviendrait à ne jamais le charger, quelle que soit la tranche. Son empreinte suffit à décider s'il a bougé, et c'est le journal qui tranche. La même règle vaut pour un lot que la fenêtre ne sait pas lire : une tranche de temps ne doit pas faire disparaître une livraison en silence.

Labo : `Labo_06_DE.md`, étape 2.

10. Oui. La fenêtre coupe le lot au premier souligné, si bien que `2023-03_bis` reste le mois de mars 2023. L'inverse le rendrait invisible à toute fenêtre, et la relivraison de mars 2023, que le Cours 03 a mise en scène, ne serait jamais rattrapée.

Labo : `Labo_06_DE.md`, étape 2.

11. La fenêtre garantit que l'exécution ne regarde que les lots de sa tranche. Le journal des chargements garantit qu'un fichier déjà chargé sous la même empreinte ne se recharge pas. La reconstruction garantit que réécrire un ensemble redonne le même ensemble.

Théorie : `Theorie_06_DE.md`, section 5.

12. Parce que son `COPY ... OVERWRITE` du Cours 02 vide le dossier avant d'écrire, et que son contrôle final compare le compte de la source et celui du bronze. Fenêtrer l'étape voudrait dire réécrire une partition parmi d'autres, et remplacer ce contrôle global par un contrôle par partition. Le coût de la reconstruction est de quelques secondes ; la garantie vaut plus cher.

Théorie : `Theorie_06_DE.md`, section 5.

13. Rater appelle la nouvelle tentative. Tarder appelle l'alerte de retard. Attendre appelle le capteur, de préférence différable, qui rend sa place pendant l'attente.

Théorie : `Theorie_06_DE.md`, section 6.

© 2026 iAlexMG. Tous droits réservés.

14. Il faut le réécrire. Airflow 3.0 a retiré les SLA, callbacks et métriques compris. Le remplacement est la Deadline Alert, déclarée sur le DAG : un point de référence, un intervalle, et un rappel enveloppé dans `SyncCallback` ou `AsyncCallback`.

Théorie : `Theorie_06_DE.md`, section 6.

15. Le pool protège une ressource partagée entre plusieurs DAG. Le nombre d'exécutions simultanées ne protège un DAG que de lui-même : il ne voit rien de ce qu'un autre DAG fait au même fichier.

Théorie : `Theorie_06_DE.md`, section 7.

16. D'abord parce qu'elle nomme le point de couture avec Analytics Engineering : quiconque lit le DAG voit où dbt entrera. Ensuite parce qu'elle fixe la place de dbt dans l'ordre, après `raw` : le jour où la tâche sera remplie, personne n'aura à se demander si le chargement doit précéder la transformation.

Théorie : `Theorie_06_DE.md`, section 8.

Questions pratiques

1. Un seul fichier, l'archive du jour, dans la fenêtre 1 :

   ```
   python -m pipeline raw --livraisons telemetrie --depuis 2026-02-02 --jusqua 2026-02-02
   ```

   Attendu : 1 fichier, 280 lignes. La télémétrie est livrée par jour, un fichier `sorties_AAAA-MM-JJ.jsonl` chacun, et le lot d'une archive est la date de son nom.

Labo : `Labo_06_DE.md`, étape 4.

2. La même commande que le témoin, avec les deux bornes sur le trimestre, dans la fenêtre 1 :

   ```
   python -m pipeline raw --livraisons erp,garanties --depuis 2026-01 --jusqua 2026-03
   ```

   Attendu sur un entrepôt qui s'arrête à décembre 2025 : 42 fichiers, 20 664 lignes. Sur un entrepôt déjà rattrapé : 0 fichier, parce que le journal les connaît tous.

Labo : `Labo_06_DE.md`, étape 9.

3. Dans l'interface web, l'entrepôt attaché :

   ```
   -- Attendu : 2026-01 14 fichiers 6 452 lignes, 2026-01-14 1 et 222, 2026-02 14 et 6 506,
   -- 2026-03 14 et 7 706, 2026-03-15 1 et 949.
   SELECT lot, COUNT(*) AS fichiers, SUM(lignes) AS lignes
   FROM entrepot.pipeline.chargements
   WHERE lot LIKE '2026-%'
   GROUP BY lot
   ORDER BY lot;
   ```


© 2026 iAlexMG. Tous droits réservés.

Les lots de mois viennent du DAG mensuel, les lots de jour du DAG quotidien et de l'essai de l'étape 4. Deux rythmes, deux formes de lot, une seule table.

Labo : `Labo_06_DE.md`, étape 10.

4. Un fichier chargé deux fois sous la même empreinte serait une ligne en double dans le journal :

```sql
-- Attendu : 0.
SELECT COUNT(*) AS doublons
FROM (
    SELECT fichier, empreinte
    FROM entrepot.pipeline.chargements
    GROUP BY fichier, empreinte
    HAVING COUNT(*) > 1
);
```

Zéro. Un même fichier peut apparaître plusieurs fois dans la table, mais chaque fois avec une empreinte différente : c'est le cas d'un export complet relivré, dont le contenu a changé.

Labo : `Labo_06_DE.md`, étape 10.

5. Les deux bornes viennent de l'intervalle, jamais de l'horloge :

```python
@dag(
    dag_id="boreal_hebdomadaire",
    schedule=CronDataIntervalTimetable("0 0 * * 1", timezone="UTC"),
    start_date=pendulum.datetime(2026, 1, 5, tz="UTC"),
    catchup=False,
    max_active_runs=1,
)
def boreal_hebdomadaire():
    BashOperator(
        task_id="raw_de_la_semaine",
        bash_command=f"{PIPELINE} raw --livraisons telemetrie "
                     f"--depuis {{{{ ds }}}} "
                     f"--jusqua {{{{ macros.ds_add(ds, 6) }}}}",
        pool=ENTREPOT,
    )
```

Trois choix, chacun avec sa raison. Le rythme : `@weekly` part le dimanche, et l'énoncé veut le lundi ; `0 0 * * 1` dit « chaque lundi à minuit », et `CronDataIntervalTimetable` en fait une tranche d'un lundi au suivant, traitée une fois finie. La borne de début : la date logique, le lundi qui ouvre la tranche, plutôt que `data_interval_start`, qui vaut la même chose pour une exécution planifiée mais part de l'heure du lancement pour une semaine rejouée à la main. La borne de fin : six jours plus tard, le dimanche, et non la fin de l'intervalle. L'intervalle se termine au début de la semaine suivante,

© 2026 iAlexMG. Tous droits réservés.

et la fenêtre du pipeline est inclusive des deux côtés : prendre `data_interval_end` ferait mordre chaque exécution d'un jour sur la suivante. Le journal des chargements empêcherait le doublon, mais le DAG mentirait sur ce qu'il fait.

Théorie : `Theorie_06_DE.md`, section 3.

6. L'étape lit `debut` et `fin` dans la section `[meteo]` de `config.toml`. Le changement consiste à prendre les bornes de la fenêtre quand elle n'est pas ouverte, et à garder la configuration comme valeur par défaut. Deux choses sont à vérifier avant : que l'API accepte une plage d'une seule journée, et ce qu'elle rend quand la journée demandée n'a pas encore été publiée. Sans cette vérification, un DAG quotidien branché sur la météo appellerait tous les matins un service qui n'a rien pour lui, et compterait cela comme un succès.

Labo : `Labo_06_DE.md`, étape 3.

7. Un capteur de fichier, placé en amont de `raw_du_mois`, qui attend l'apparition de `sources/erp/extraits/{{ logical_date.strftime('%Y-%m') }}/VBAK.csv`. Il doit être différable, faute de quoi il occupe une place d'exécution pendant toute son attente, qui peut durer des heures. Il ne prend pas le pool `entrepot` : il ne touche pas à l'entrepôt.

Théorie : `Theorie_06_DE.md`, section 6.

8. Dans la fenêtre 2, l'Ubuntu ouvert dans `plateforme` :

   ```
   docker compose exec airflow-scheduler airflow pools list
   docker compose exec airflow-scheduler airflow pools set entrepot 2 "Deux places"
   ```

   Avec deux places, les deux DAG pourraient écrire l'entrepôt en même temps, et la seconde tâche recevrait `Could not set lock on file ... Conflicting lock is held`. Le pool ne crée pas la contrainte, il la respecte : la limite d'un seul écrivain vient de DuckDB, pas d'Airflow.

Labo : `Labo_06_DE.md`, étape 7.

Diagnostics

1. `$(date +%F)` est évalué par l'interpréteur de commandes au moment où la tâche tourne : il donne la date du jour, pas celle de l'intervalle. Les trois exécutions rattrapées ont donc demandé la même journée, celle du rattrapage, et chargé le même lot. Le DAG n'a jamais raté parce qu'il n'y avait rien d'anormal à signaler : il a fait exactement ce qui était écrit. Le correctif est `{{ ds }}`, substitué par Airflow avant le lancement.

Théorie : `Theorie_06_DE.md`, section 3.

2. D'abord la date de début, qui remonte trop loin : l'historique de Boréal a été chargé d'un bloc au Cours 05, et le DAG n'a pas à le refaire. Ensuite l'absence de limite sur les exécutions simultanées : `max_active_runs=1` les met en file plutôt qu'en concurrence, ce qui compte doublement ici puisqu'elles écrivent toutes le même fichier DuckDB.

Théorie : `Theorie_06_DE.md`, section 4.

© 2026 iAlexMG. Tous droits réservés.

3. Parce que la nouvelle tentative traite le symptôme. La cause est qu'un autre DAG écrit le même fichier au même moment, et rien ne l'empêche de recommencer. Un jour sur trois devient un jour sur deux quand une tâche s'allonge, et le second essai finit par tomber aussi. La réponse est le pool `entrepot` d'une seule place, qui sérialise les écritures. La nouvelle tentative reste utile, mais comme filet, pas comme mécanique.

Théorie : `Theorie_06_DE.md`, section 7.

4. Dans cet ordre : `airflow dags list-import-errors`, qui nomme le fichier, la ligne et l'exception ; puis les journaux du lecteur de DAG, `docker compose logs airflow-dag-processor` ; puis seulement le fichier lui-même. L'erreur la plus fréquente de ce cours est `Provided callback ... is not awaitable`, quand `AsyncCallback` reçoit une fonction ordinaire au lieu d'une coroutine. Le fichier est correct au sens de Python : il s'importe, mais la déclaration du DAG lève.

Labo : `Labo_06_DE.md`, étape 8.

5. La fenêtre n'est pas en cause, et le DAG mensuel non plus. La télémétrie appartient au DAG quotidien, qui a `catchup=False` : il ne rattrape pas les journées de 2026 qui précèdent sa mise en route. Le partage des livraisons a rendu visible un trou qui existait déjà, caché par le fait que la fenêtre du mois emportait aussi les archives quotidiennes. La réponse est de charger ces journées une fois, à la main, avec une fenêtre large sur la télémétrie, puis de laisser le DAG quotidien suivre.

Labo : `Labo_06_DE.md`, étape 3.

6. Dans une nouvelle tentative. Une tâche a échoué une première fois, et le délai de reprise est de cinq minutes : l'exécution a donc duré cinq minutes de plus, puis a réussi au second essai, et le DAG est vert. Les journaux de la tâche montrent bien deux tentatives ; c'est la première qu'il faut lire, la seconde n'a rien à dire.

Labo : `Labo_06_DE.md`, étape 8.

Questions à développer

A. Une panne s'annonce : un rouge dans l'interface, une alerte, un courriel. Quelqu'un regarde. Une erreur d'intervalle ne s'annonce pas : la tâche réussit, le compte de lignes est plausible, et la donnée est fausse. Le coût n'est pas l'arrêt, c'est la confiance : le jour où on s'en aperçoit, on ne sait plus quelles journées sont bonnes, et il faut tout recharger pour en être sûr. Deux habitudes la rendent improbable. La première est de bannir l'horloge des commandes de tâches, sans exception, et de traiter `date`, `now()` et `datetime.today()` dans un DAG comme une faute de revue. La seconde est de rejouer une exécution ancienne, volontairement, après chaque changement du DAG : si le résultat n'est pas identique, quelque chose lit l'horloge.

Théorie : `Theorie_06_DE.md`, section 3.

B. Les comptes de lignes laissent passer toute erreur qui conserve le nombre. Cas concret : le DAG mensuel charge les extraits de janvier, février et mars, mais une commande mal écrite met le lot de février sur les lignes de mars et l'inverse. Chaque table a le même nombre de lignes des deux côtés, chaque mois a son compte. Pourtant la colonne `_lot` ment, et toute analyse qui découpe par mois donnera des chiffres faux, sans que rien ne le signale. La comparaison dans les deux sens attrape ce cas, parce qu'elle compare des lignes entières, colonne de lignage comprise.

© 2026 iAlexMG. Tous droits réservés.

Une précision qui change la conclusion : il faut `EXCEPT ALL`, et non `EXCEPT`. `EXCEPT` compare des ensembles, donc il écarte les exemplaires en double avant de comparer ; un mois chargé deux fois lui serait invisible, puisque les lignes en trop sont identiques à celles du témoin. Seul `EXCEPT ALL` compte les exemplaires. Le script de l'étape 10 s'en sert, et compare aussi les deux comptes de lignes, parce qu'une preuve qui ne peut pas échouer ne prouve rien.

Labo : `Labo_06_DE.md`, étape 10.

C. La proposition ne tient pas, et chacune des trois mécaniques couvre un cas que les autres ne voient pas. Sans le journal, deux fenêtres qui se chevauchent doubleraient les lignes : rejouer le trimestre après avoir chargé février donnerait février deux fois. Sans le journal encore, un export complet relivré ne serait jamais reconnu comme ayant changé, puisqu'il n'a pas de date. Sans la reconstruction, le bronze devrait tenir un journal par partition et un contrôle par partition, pour une étape qui se refait en trente secondes. Et sans la fenêtre, le rattrapage perdrait son sens : la première exécution avalerait tout. Ce qu'on peut concéder à l'architecte, c'est que trois mécaniques demandent d'être expliquées, et que c'est le rôle de la documentation du paquet, pas une raison d'en retirer une.

Théorie : `Theorie_06_DE.md`, section 5.

Mises en situation

1. Trois choses à soulever. Un DAG qui écrit toutes les quinze minutes prendra le pool `entrepot` quatre fois par heure, et bloquera d'autant les chargements quotidien et mensuel : il faut regarder combien de temps il tient le fichier. Ensuite, l'étape `bronze` reconstruit tout, ce qui prend une trentaine de secondes : à ce rythme, la machine passerait son temps à réécrire les mêmes partitions. Enfin, quinze minutes n'est pas un rythme de cold path. La proposition est de séparer les deux chemins : ce qui doit être frais passe par le hot path du Cours 09, et le cold path garde son rythme quotidien. Si le besoin est réellement un rafraîchissement d'un quart d'heure sur des fichiers, alors la question porte sur le format de l'entrepôt, pas sur l'orchestrateur.

Théorie : `Theorie_06_DE.md`, sections 5 et 7.

2. On laisse l'orchestrateur faire. Le DAG mensuel rattrape les intervalles manqués, et la preuve du labo dit exactement ce que cela vaut : trois exécutions successives donnent la même `RAW`, ligne par ligne, qu'un chargement d'un bloc. Le DAG quotidien, lui, ne rattrape pas : il faudra charger les journées manquées une fois, à la main, avec une fenêtre sur ces trois jours. Ce qui permet d'en être sûr n'est pas la confiance dans Airflow, c'est la comparaison faite à l'étape 10 et le test qui la refait en une demi-seconde.

Théorie : `Theorie_06_DE.md`, section 4.

3. On regarde d'abord lesquelles de ces quinze tâches touchent la même chose : deux tâches qui écrivent des tables différentes, à partir de sources différentes, n'ont aucune raison de s'attendre. On regarde ensuite les durées, pour voir ce que la mise en file coûte réellement. Puis on lui dit qu'une dépendance qui n'existe pas coûte du temps à chaque exécution et masque les vraies : le jour où une tâche du milieu échouera, il ne saura pas si les suivantes auraient pu tourner. La question à lui poser, tâche par tâche, est simple : celle-ci échouerait-elle, ou mentirait-elle, si la précédente n'avait pas fini ?

Théorie : `Theorie_06_DE.md`, section 2.

© 2026 iAlexMG. Tous droits réservés.

4. Le délai se choisit sur la distribution observée, pas sur la moyenne. Huit minutes d'ordinaire, quarante une fois : un délai de trente minutes laisse passer le cas normal et le cas lent connu, et signale ce qui sort des deux. À dix minutes, l'alerte partirait plusieurs fois par mois sans qu'il y ait rien à faire, et une alerte qu'on apprend à ignorer ne sert plus à rien. C'est ce qu'il faut répondre : une alerte de retard n'est pas une mesure de performance, c'est un signal qu'on veut voir rarement et prendre au sérieux à chaque fois. Si la direction veut suivre les huit minutes, c'est la table `pipeline.executions` qui répond, pas une alerte.

Théorie : `Theorie_06_DE.md`, section 6.

5. Première option, `dbt build` reste une tâche du DAG quotidien : la chaîne se lit d'un coup, de la source aux tables livrées, et l'ordre est garanti sans rien de plus. Prix à payer : une erreur de modèle dbt fait échouer le DAG de chargement, et les deux équipes partagent un seul calendrier. Seconde option, un DAG à part, déclenché par un jeu de données qu'écrit `raw` : chaque équipe garde son DAG et son rythme, et dbt peut tourner plusieurs fois par jour sans rejouer le chargement. Prix à payer : deux graphes à lire pour comprendre une seule chaîne, et un mécanisme de déclenchement de plus à connaître. La recommandation est de commencer par la première, tant que les deux rythmes sont les mêmes, et de passer à la seconde le jour où ils divergent. La tâche vide posée dans ce cours rend ce passage facile : c'est une ligne à remplacer, pas un DAG à refaire.

Théorie : `Theorie_06_DE.md`, section 8.

© 2026 iAlexMG. Tous droits réservés.